Particle GFlowNets: Rethinking Generative Marginalization Models

Tiago da Silva (∗), Diego Mesquita (+), Salem Lahlou (∗)

Keywords — GFlowNets, Markov chain Monte Carlo

TL;DR
- Generative Marginalization Models (MaMs) are fast autoregressive samplers for discrete distributions over combinatorial spaces;
- we show MaMs can be seen as an instance of a GFlowNet with a newly defined learning objective;
- we extend this objective to non-autoregressive discrete modelling, showcasing for the first time how to effectively train GFlowNets **without sampling entire trajectories** for each gradient step;
- we demonstrate our algorithm—called **Particle GFlowNets**—significantly accelerates learning convergence in terms of wall-clock time.

I. Background: Sampling from Discrete Distributions

We have a finite set $\mathcal{X}$ of states and a positive function $R : \mathcal{X} \to \mathbb{R}_+$. Our objective is to sample from

$$x \sim \pi(x) \propto R(x) \tag{1}$$

Assumption. $\mathcal{X}$ is a **combinatorial space**. This means each $x \in \mathcal{X}$ can be described as a **set of components** from a fixed set $\mathcal{C}$.

Examples include the space of **graphs**, which can be described as a set of edges, and the space of **sets**.

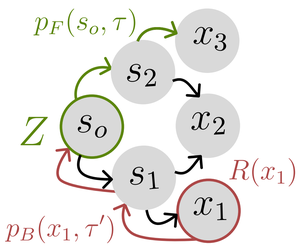


Figure 1: A GFlowNet **constructs** $x \in \mathcal{X}$ in proportion to the **target function** $R(x)$.

A GFlowNet learns a **policy function** p_F that builds each $x \in \mathcal{X}$ such that

$$\sum_{\tau : s_o \rightsquigarrow x} p_F(s_o, \tau) = \frac{R(x)}{Z} \tag{2}$$

while also estimating

$$Z = \sum_{x \in \mathcal{X}} R(x). \tag{3}$$

We paramterize $p_{F(s,\cdot)}$ with a neural network.

II. MaMs as GFlowNets

A Generative Marginalization Model (MaM) assumes the generative process is **autoregressive**.

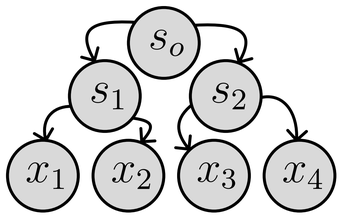


Figure 2: Autoregressive.

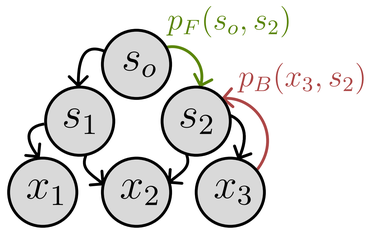


Figure 3: Non-autoregressive.

We think of ***d*-sized sequences** with token set $\mathcal{C}$, a combinatorial space on $\mathcal{C} \times [d]$. A MaM's goal is to ensure

$$F(s)p_F(s, s') = F(s') \text{ and } F(x) = R(x) \tag{4}$$

for $x \in \mathcal{X}$ and **adjacent** states s, s', i.e., for which $|s' \setminus s| = 1$. Example: $s = \{(c, 1)\}$ and $s' = \{(c, 1), (c', 2)\}$.

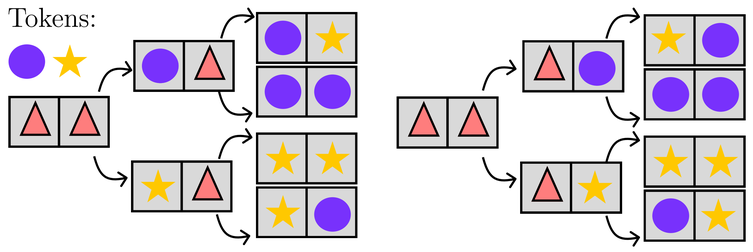


Figure 4: A Permutation (σ) Conditioned generative process (PCGP): elements added according to σ.

MaMs maintain B **Markov chains** $(s_{b,t})_{t \geq 1}$ for $b \in \{1, ..., B\}$ via **Markov kernels** compatible with any PCGP of the kind described in Figure 4 and minimize

$$\mathcal{L}(p_F, F, t) = \sum_{1 \leq b \leq B} \left(\log \frac{F(s_{b,t}) p_F(s_{b,t}, s_{b,t+1})}{F(s_{b,t+1})} \right)^2 \tag{5}$$

Proposition. (MaMs are **PC GFlowNets**). For each MaM abiding by Equation 4 and each PCGP in Figure 4, there is a unique GFlowNet satisfying Equation 2. We call **PC GFlowNets** the set of all such GFlowNets.

III. Particle GFlowNets: Learning GFlowNets from Partial Trajectories

We extend MaMs to **non-autoregressive generative processes** by simply introducing a **backward policy**, i.e.,

$$\mathcal{L}_P(p_F, p_B, F, t) = \sum_{1 \leq b \leq B} \left(\log \frac{F(s_{b,t}) p_F(s_{b,t}, s_{b,t+1})}{F(s_{b,t+1}) p_B(s_{b,t+1}, s_{b,t})} \right)^2.$$

The Markov chains $(s_{b,t})_{t \geq 1}$ are constructed by **backward-forward kernels**, as described in the next figure.

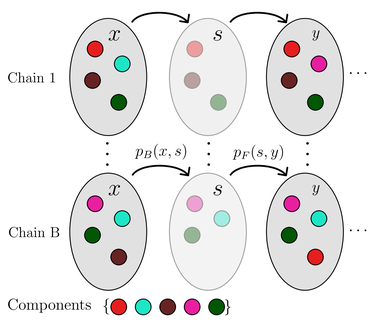


Figure 5: Markov chain for **Particle GFlowNets**.

The main process lives on $\mathcal{X}$: $(x_{b,t})_{t \geq 1}$. We sample $l \sim \text{Poisson}(\log L)$, and let

$$s_{b,t} \sim p_B^{(k)}(x_{b,t}, \cdot)$$

$$s_{b,t+1} \sim p_F(s_{b,t}, \cdot).$$

(L is the maximum trajectory size). We then sample

$$x_{b,t+1} \sim p_F^{(k-1)}(s_{b,t+1}, \cdot).$$

We evaluate $\mathcal{L}_P$ for both transitions, $x_{b,t} \to x_{b,t+1}$ and $s_{b,t} \to s_{b,t+1}$. Our model enables both fast, correlated and expensive, independent sampling from $R(x)$.

Proposition. (Sampling correctness). When $\mathcal{L}_P$ is **minimized**, the GFlowNet satisfies Equation 2 and the **Markov chain** $(x_{b,t})_{t \geq 1}$ is **ergodic** relatively to $R(x)$.

IV. Experiments

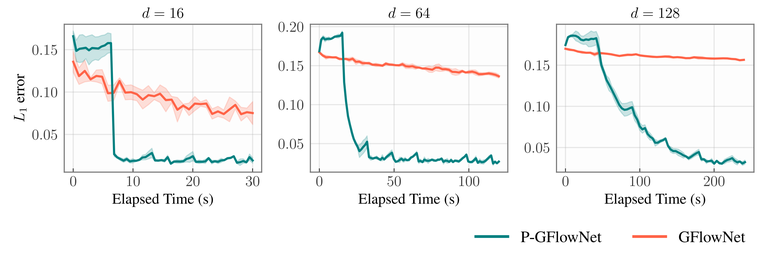


Figure 6: Learning curve set generation (d = set size).